

Sparse PCA

Iago Zagnoli Albergaria

May 2026

1 Metodologia e Implementação

A implementação do algoritmo de **SPCA** seguiu estritamente os elementos descritos na documentação da atividade, nos slides apresentados em aula e no artigo do Bertsimas.

1.1 Cálculo de Bounds

- **Upper Bounds** (ub_1, ub_2, ub_3): O limite superior é definido como o mínimo entre três limites discutidos em sala. O ub_1 é obtido pelo autovalor máximo da submatriz restrita Q_u . O ub_2 utiliza o traço da matriz, somando os k maiores elementos das diagonais permitidas. O ub_3 é baseado no Teorema dos Círculos de Gershgorin, maximizando as k maiores entradas em módulo por coluna.
- **Lower Bounds** (lb_1, lb_2): Para os limites inferiores, foi necessário implementar o Método da Potência Truncada, que intercala multiplicações sucessivas pela matriz Q com o truncamento do vetor, mantendo apenas as k variáveis de maior magnitude que respeitam as restrições do nó.

1.2 Estratégia de Branching e Seleção de Nós

A estratégia de seleção de um nó na lista de nós a serem explorados é híbrida, ou seja, usa tanto a estratégia de Best-first quanto a de Depth-first. O algoritmo de Best-First escolhe explorar o nó com o maior limite superior. Já o algoritmo de Depth-First escolhe explorar o último nó adicionado na lista. Na seleção híbrida, usamos Best-first enquanto o tamanho da lista de nós a serem explorados não ultrapassa um limite fixo (definido como 50 na descrição da atividade). Caso o tamanho ultrapasse, o algoritmo muda para Depth-first. Além disso, vale ressaltar que os nós explorados não são automaticamente apagados da lista para prevenir que a lista seja constantemente redimensionada. Inicialmente, quando um nó é processado, ele é marcado como *morto* em um vetor auxiliar e não é mais visitado. Quando o tamanho da lista de nós ultrapassa 5000 elementos, todos os nós marcados como mortos são deletados. Essa decisão foi tomada

como inspiração no código em *julia* disponível no artigo estudado, onde o autor também fazia essa deleção *preguiçosa*.

Como pedido na descrição da atividade, a escolha da variável para executar o branching em um nó é feita observando a coordenada livre que apresenta a maior magnitude no autovetor principal da matriz restrita Q_u , já que a mesma indica a direção de maior impacto na variância.

2 Resultados

Esta seção apresenta os resultados obtidos com a execução do algoritmo para os diferentes conjuntos de dados.

2.1 Definição das Métricas

Para facilitar a interpretação dos resultados, as tabelas abaixo utilizam as seguintes métricas:

- **K:** número máximo de features não nulas por componente.
- **Var. Total (%):** Porcentagem da variância total acumulada explicada pelo conjunto de componentes extraídos até o fim do algoritmo (critério de parada de 70%).
- **Comps:** Número de componentes principais esparsos necessários para atingir o critério de variância.
- **Tempo (s):** Tempo total de execução do processo em segundos.
- **Nós Explorados:** Quantidade total de nós processados na árvore de busca durante todas as execuções do Branch-and-Bound.
- **Gap Médio (%):** Média dos gaps de otimalidade obtidos ao final de cada extração de componente.

2.2 Datasets Pitprops e normWine

Table 1: Desempenho Geral - Pitprops e normWine

Dataset	K	Var. Total (%)	Comps	Tempo (s)	Nós Explorados	Gap Médio (%)
Pitprops	5	76.21	5	0.28	95	10.31
Pitprops	10	73.31	4	0.14	47	0.95
normWine	5	71.43	4	0.12	57	11.18
normWine	10	72.79	4	0.22	63	1.36

Como as matrizes de covariância tem baixa dimensionalidade ($n = 13$), o algoritmo alcança o critério de parada rapidamente e sem a necessidade de explorar

muitos nós. O aumento da cardinalidade K de 5 para 10 também resultou em uma convergência mais eficiente (com gaps médios caindo para cerca de 1%) e reduziu ligeiramente o número de componentes necessários para atingir o limiar de 70% de variância acumulada.

2.3 Dataset: Spambase ($n = 57$)

Table 2: Desempenho Geral - Spambase

K	Var. Total (%)	Comps	Tempo (s)	Nós Explorados	Gap Médio (%)
3	70.01	31	44.41	4 855	29.12
4	70.07	30	205.12	22 166	22.57
5	70.51	30	926.17	100 284	20.52
6	70.86	30	3 682.96	349 360	16.96
7	71.02	30	7 932.09	760 009	14.84
8	71.32	30	27 993.46	2 1684 80	13.46

Os resultados do *Spambase* evidenciam a complexidade exponencial do algoritmo Branch-and-Bound. Conforme a quantidade de features a serem escolhidas aumenta, o tempo de processamento e a quantidade de nós explorados crescem exponencialmente, indo de 44 segundos com $K = 3$ para quase 8 horas com $K = 8$. Apesar disso, ao permitir a seleção de mais variáveis por componente, o limitante primal ganha força, o que explica a queda do gap médio de otimalidade de 29,12% ($K = 3$) para 13,46% ($K = 8$).

2.4 Dataset: Colon Microarray

Para o dataset Colon, os resultados consideram o subconjunto das $n \in \{80, 100, 120\}$ features com maior variância, conforme o filtro solicitado na documentação do trabalho.

Table 3: Desempenho Geral - Colon Microarray

n	K	Var. Total (%)	Comps	Tempo (s)	Nós Explorados	Gap Médio (%)
80	5	70.92	20	376.29	58 961	55.29
100	5	71.18	24	3 795.40	530 684	62.72
120	5	70.75	29	10 626.65	1 122 043	64.89

Os resultados do dataset *Colon Microarray* mostram um impacto direto do tamanho do filtro de variância (n) na complexidade computacional do algoritmo. Ao aumentar o número de genes selecionados, mesmo que a cardinalidade fixa em $K = 5$ seja mantida, observa-se um crescimento significativo no esforço computacional para um acréscimo de apenas 20 variáveis.

Esse fenômeno é reflexo direto da expansão do espaço de busca combinatória na árvore do *Branch-and-Bound*, como evidenciado pelo número de nós explorados, que também salta enormemente. Além disso, o gap médio de otimalidade aumenta de 55,29% para 62,72%. Esse distanciamento indica que a inclusão de novos genes introduz maior redundância na matriz de covariância Q , tornando as relaxações duais menos restritivas e dificultando a poda precoce de nós na árvore de busca. Por fim, o fato das configurações com maior quantidade de features exigir mais componentes para atingir o patamar de 70% de variância demonstra que a dispersão da informação aumenta com o tamanho do filtro, diluindo a capacidade dos primeiros componentes esparsos de reterem a variância global dos dados.

3 Análise de Sobreposição (Matriz J) e Variância acumulada

Nesta seção, observamos a qualidade dos componentes extraídos por dois pontos de vista que se complementam. Primeiro, a matriz J mostra o quanto os suportes dos componentes se sobrepõem. Quando esses valores são menores, isso indica que o algoritmo conseguiu separar melhor as variáveis, deixando os vetores menos parecidos entre si. Depois, as curvas de variância mostram o quanto de informação cada novo componente consegue trazer ao longo das iterações de deflação, até chegar ao limite de 70% da variância total.

3.1 Pitprops

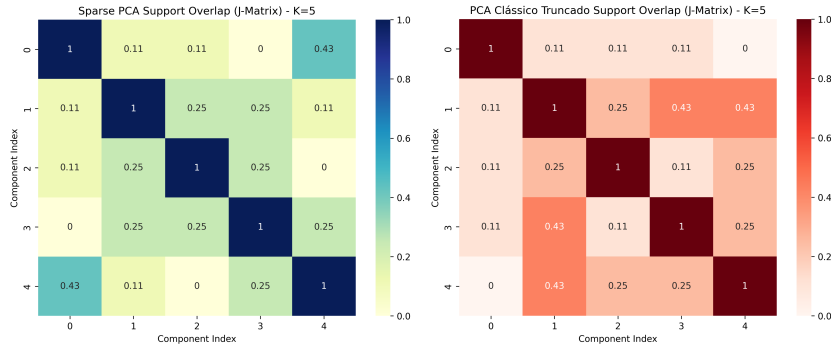


Figure 1: Matrizes J para Pitprops ($K = 5$). À esquerda, matriz do PCA Esperso Ótimo. À direita, matriz do PCA Clássico Truncado.

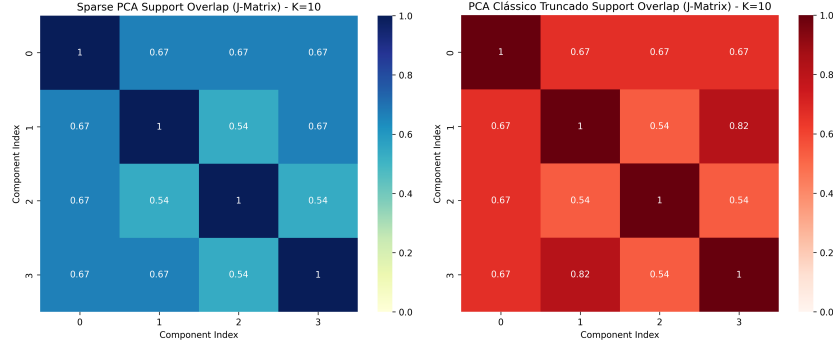


Figure 2: Matrizes J para Pitprops ($K = 10$). À esquerda, matriz do PCA Esperso Ótimo. À direita, matriz do PCA Clássico Truncado.

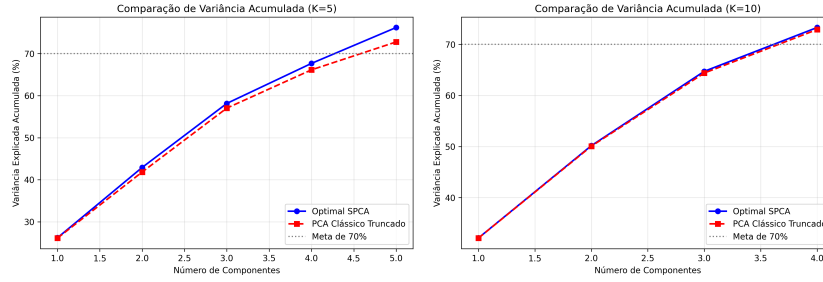


Figure 3: Comparação de variância acumulada (SPCA vs Clássico Truncado) para Pitprops com $K = 5$ (esquerda) e $K = 10$ (direita).

No dataset *Pitprops*, os componentes gerados pelo *Optimal SPCA* mostram menos sobreposição e matrizes J mais esparsas do que o PCA clássico truncado. Isso sugere que o método consegue separar melhor as variáveis e evitar que os mesmos padrões apareçam em vários componentes. Nas curvas de variância, o SPCA também se sai ligeiramente melhor, acumulando um pouco mais de variância com o mesmo número de componentes. Já o truncamento direto, por simplesmente zerar coeficientes depois da extração, acaba perdendo parte importante da estrutura dos dados.

3.2 normWine

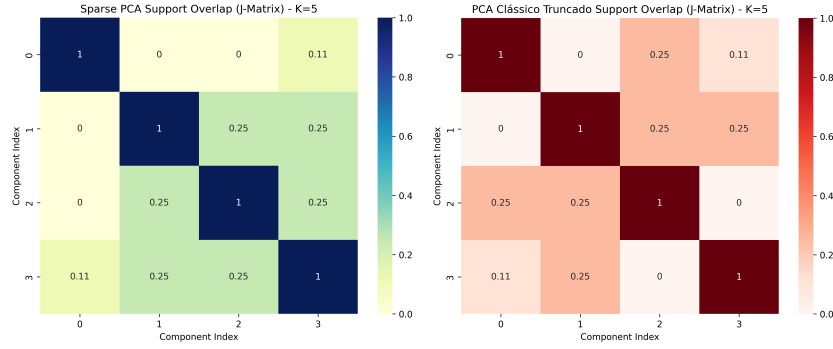


Figure 4: Matrizes J para normWine ($K = 5$). À esquerda, matriz do PCA Espasso Ótimo. À direita, matriz do PCA Clássico Truncado.

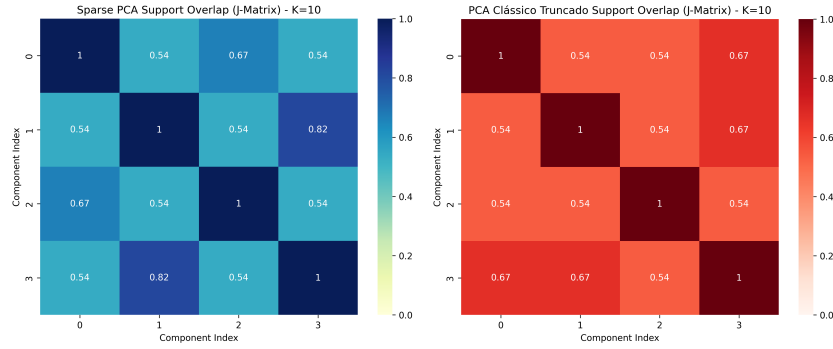


Figure 5: Matrizes J para normWine ($K = 10$). À esquerda, matriz do PCA Espasso Ótimo. À direita, matriz do PCA Clássico Truncado.

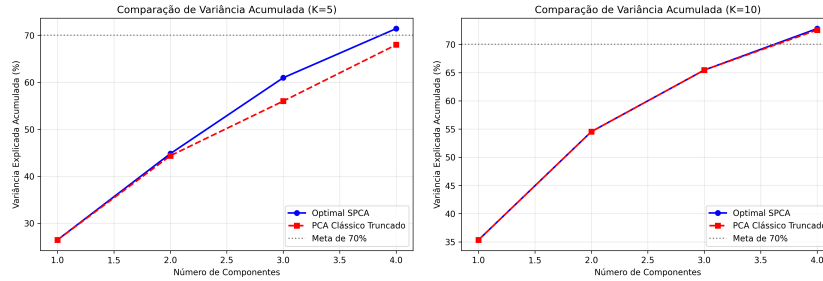


Figure 6: Comparação de variância acumulada (SPCA vs Clássico Truncado) para normWine com $K = 5$ (esquerda) e $K = 10$ (direita).

No *normWine*, o comportamento é bem parecido com o observado em *Pitprops*. As matrizes J indicam que o *Branch-and-Bound* ajuda a reduzir redundâncias e gera suportes mais distintos entre si. Isso faz com que o SPCA trabalhe de forma mais eficiente do que o truncamento clássico. Nas curvas de variância, o algoritmo exato também mantém vantagem, mostrando que otimizar diretamente sob a restrição de cardinalidade leva a resultados mais consistentes do que aplicar a esparsidade depois, de forma forçada.

3.3 Spambase

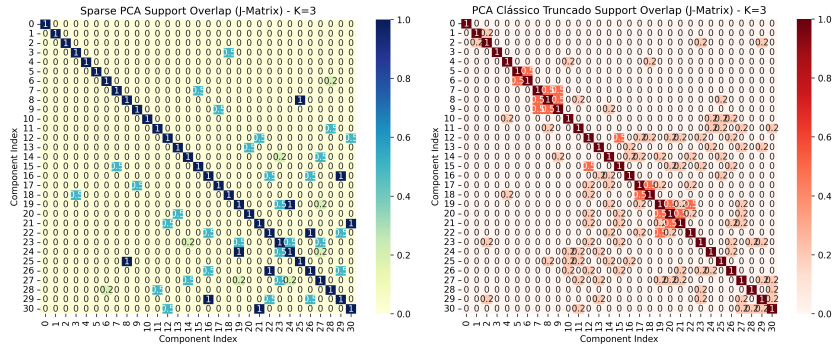


Figure 7: Matrizes J para Spambase ($K = 3$). À esquerda, matriz do PCA Esperso Ótimo. À direita, matriz do PCA Clássico Truncado.

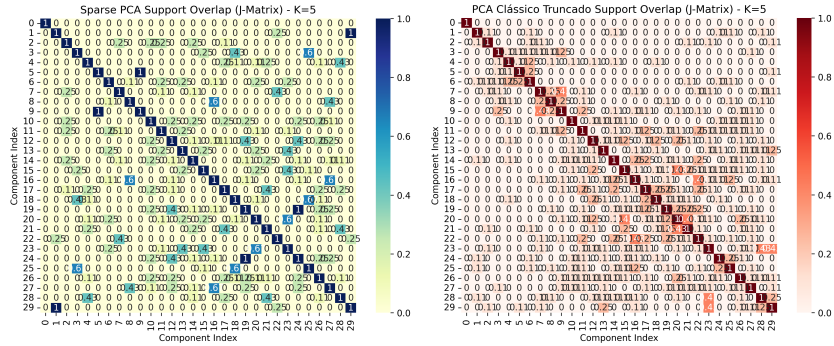


Figure 8: Matrizes J para Spambase ($K = 5$). À esquerda, matriz do PCA Esperso Ótimo. À direita, matriz do PCA Clássico Truncado.

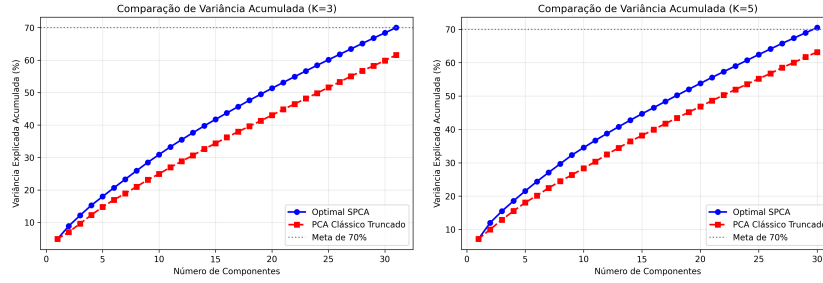


Figure 9: Comparação de variância acumulada (SPCA vs Clássico Truncado) para Spambase com $K = 3$ (esquerda) e $K = 5$ (direita).

No *Spambase*, a diferença entre os métodos fica ainda mais clara. Os *heatmaps* da matriz J do PCA clássico truncado mostram bastante sobreposição entre os componentes, o que indica que ele acaba reutilizando variáveis parecidas em mais de uma direção. Já o SPCA, por usar a deflação da matriz Q de forma mais cuidadosa, consegue explorar melhor novas variáveis e manter os suportes mais esparsos. Isso também aparece nas curvas de variância: o SPCA cresce mais rápido e alcança a meta de 70%, enquanto o PCA truncado fica para trás e para em torno de 60%, usando o mesmo número de componentes.

3.4 Colon Microarray

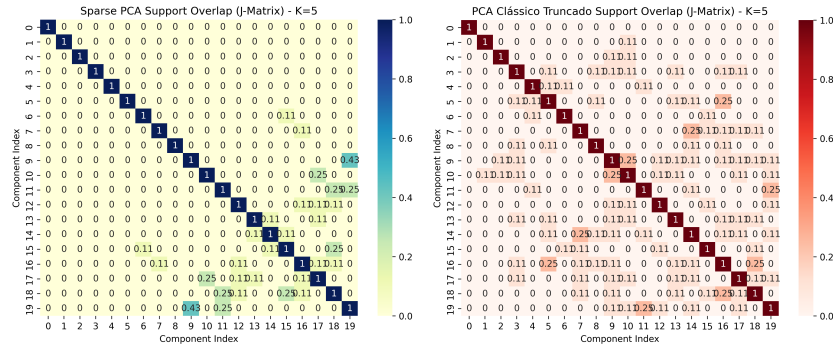


Figure 10: Matrices J for Colon Microarray ($n = 80, K = 5$). À esquerda, matriz do PCA Esperso Ótimo. À direita, matriz do PCA Clássico Truncado.

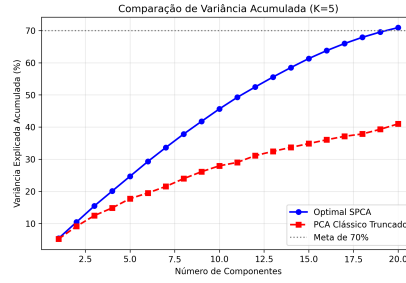


Figure 11: Comparação de variância acumulada entre SPCA e PCA Clássico Truncado para Colon Microarray ($n = 80, K = 5$).

O *Colon Microarray* mostra bem a diferença entre os métodos em um cenário de alta dimensionalidade. No SPCA, a matriz J sugere que genes relevantes estão sendo capturados de forma mais limpa e com menos repetição entre componentes. Já no PCA clássico truncado, a perda de coeficientes na hora do truncamento afeta bastante a representação dos dados. Isso aparece nas curvas de variância, onde o SPCA consegue acumular muito mais informação com o mesmo número de componentes, enquanto o PCA truncado fica bem mais limitado e não representa os dados tão bem.